

Algorithms and Data structures

Assignment Report

Student Name; Joel Manoj John

Student Number; C24358441

Module: CMPU2001

1. Introduction

The report goes through 4 java implemented graph algorithms and these algorithms were tested on a weighted undirected graph with 22 edges and 13 vertices. The following graph algorithms are;

- Breadth First Transversal (BF) = this explored level by level using a queue and then finds the shortest distance from the source.
- Depth First Transversal (DF) = goes through the graph as deep as possible before backtracking and uses Cormen's grey/white/black colouring with timestamps.
- Prim's MST = makes a minimum spanning tree (MST) one vertex at a time and always adds the smallest edge available with a custom minimum heap.
- Krustal's MST = this starts by sorting all the edges by weight then adds them up one by one then skips any that would form a cycle using Union Find.

The graph is stored as an adjacency list which is an array of linked lists, one per vertex, which is way more space efficient than a usual matrix.

2. Adjacent list representation

Each of these vertices store a linked list of the vertices beside them and edge weights. Edges are inserted at the head of the lists.

Vertex	Adjacency list
A	F(2) -> G(6) -> B(1) -> NULL
B	E(4) -> D(2) -> C(1) -> A(1) -> NULL
C	E(4) -> B(1) -> NULL
D	F(1) -> E(2) -> B(2) -> NULL
E	L(4) -> F(2) -> G(1) -> D(2) -> C(4) -> B(4) -> NULL
F	L(2) -> E(2) -> D(1) -> A(2) -> NULL
G	L(5) -> J(1) -> H(3) -> E(1) -> A(6) -> NULL
H	I(2) -> G(3) -> NULL
I	K(1) -> H(2) -> NULL

J	K(1) -> l(3) -> M(2) -> G(1) -> NULL
K	J(1) -> l(1) -> NULL
L	M(1) -> J(3) -> G(5) -> F(2) -> E(4) -> NULL
M	L(1) -> J(2) -> NULL

3. Prims MST, Step by Step (starting with L (12))

Prims algorithm works by growing one spanning tree from the starting and/or source vertex, always adding the smallest/cheapest edge that connects a new vertex to the existing tree. In this case where we start with L the cheapest known edge which is l is initialised to 0 and all the others are assigned to infinity below is the step by step execution of the program;

steps	extract	MST edge	cost	best cost updates
1	l	source	0	M = 1 by L, J = 3 by l, g = 5 by l, f = 2 by l, e = 4 by l
2	m	L to m	1	J improved to 2 by m
3	f	L to f	2	E improved to 2 by f, d = 1 by f, a = 2 by f
4	d	F to d	1	B = 2 by d
5	j	M to j	2	K = 1 by j, g improved to 1 by j
6	k	J to k	1	l = 1 by k
7	g	J to g	1	H = 3 by g, e improved to 1 by g
8	i	K to l	1	H improved to 2 by i
9	e	G to e	1	C = 4 by e
10	b	D to b	2	C improved by 1 by b, a improved by 1 by b
11	c	B to c	1	No improvements
12	a	B to a	1	No improvements
13	h	l to h	2	Mst finished

Total MST weight adds up to 16.

4. Kruskal's MST, Step by Step

Kruskal's algorithm works by sorting all the edges by weight cheapest first (quicksort) then goes through the list one at a time then if there not yet connected it connects it if not skips and the union find tracks which of these vertices are connected as it like a bookkeeping system. For each of the edges if the end points are in the same component, it would reject as it would form a cycle or if the end points were in different components it would add. Below would be the sorted edges from smallest to largest and the step by step union find sets;

Sorted edges :

Weight 1: A-B B-C D-F E-G G-J I-K J-K L-M

Weight 2: A-F B-D D-E E-F F-L H-I J-M

Weight 3 and above: G-H J-L B-E C-E E-L G-L A-G

Step	Edge	Weight	Add or Reject	Union find sets after this step
1	A to b	1	Add	{A,B} {C} {D} {E} {F} {G} {H} {I} {J} {K} {L} {M}
2	B to c	1	Add	{A,B,C} {D} {E} {F} {G} {H} {I} {J} {K} {L} {M}
3	D to f	1	Add	{A,B,C} {D,F} {E} {G} {H} {I} {J} {K} {L} {M}
4	E to g	1	Add	{A,B,C} {D,F} {E,G} {H} {I} {J} {K} {L} {M}
5	G to j	1	Add	{A,B,C} {D,F} {E,G,J} {H} {I} {K} {L} {M}
6	I to k	1	Add	{A,B,C} {D,F} {E,G,J} {H} {I,K} {L} {M}
7	J to k	1	Add	{A,B,C} {D,F} {E,G,I,J,K} {H} {L} {M}
8	L to m	1	Add	{A,B,C} {D,F} {E,G,I,J,K} {H} {L,M}
9	A to f	2	Add	{A,B,C,D,F} {E,G,I,J,K} {H} {L,M}
10	E to f	2	Add	{A,B,C,D,E,F,G,I,J,K} {H} {L,M}
11	F to l	2	Add	{A,B,C,D,E,F,G,I,J,K,L,M} {H}
12	B to d	2	Reject	B and D in same set this would create a cycle

13	H to i	2	Add	{A,B,C,D,E,F,G,H,I,J,K,L,M} MST complete
----	--------	---	-----	---

Total MST weight = 16 which is the same as Prim's algorithm.

5. MST Diagrams

Both algorithms produced the same total weight 16, the two diagrams below show each algorithms result on the graph

Diagram 1, Prim's MST (green, starting from L)

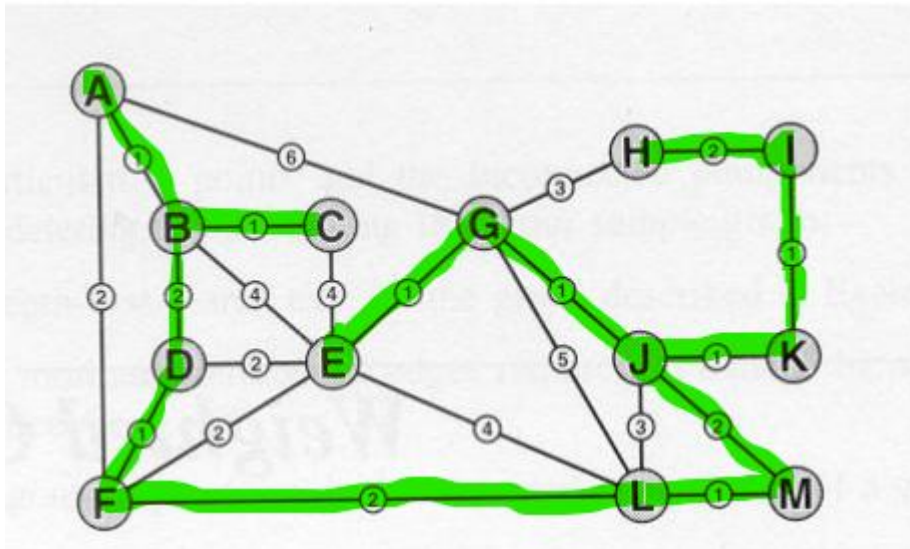
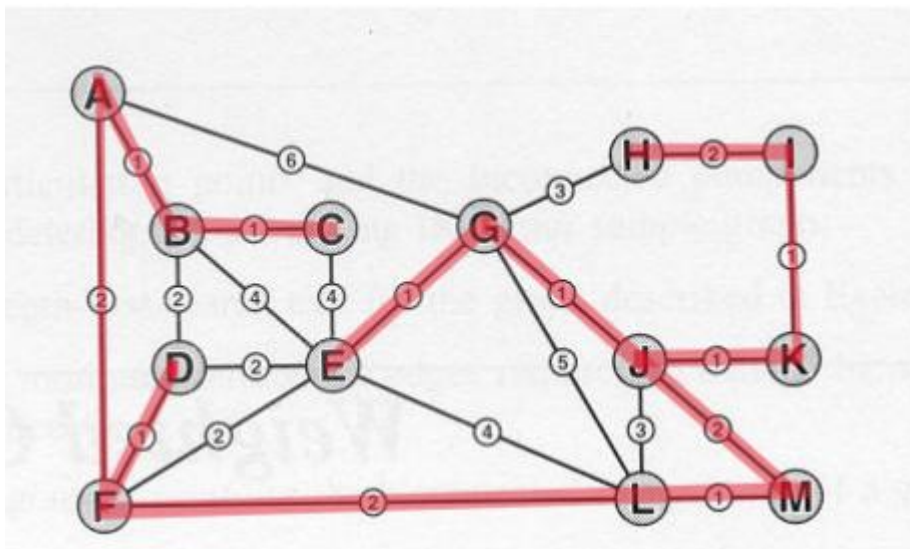


Diagram 2, Kruskal's MST (red)



6. Program Output

Console outputs running both programs with Wgraph1.txt, the user types the filename and start vertex when prompted, it autodetects the file format automatically

GraphAdjList.java:

Enter graph filename: Wgraph1.txt

Enter starting vertex number (12 for L): 12

Parts[] = 13 22 [plain format]

Reading edges from text file

Edge A--(1)--B

Edge A--(6)--G

Edge A--(2)--F

Edge B--(1)--C

Edge B--(2)--D

Edge B--(4)--E

Edge C--(4)--E

Edge D--(2)--E

Edge D--(1)--F

Edge E--(1)--G

Edge E--(2)--F

Edge E--(4)--L

Edge F--(2)--L

Edge G--(3)--H

Edge G--(1)--J

Edge G--(5)--L

Edge H--(2)--I

Edge I--(1)--K

Edge J--(2)--M

Edge J--(3)--L

Edge J--(1)--K

Edge L--(1)--M

adj[A] -> |F | 2| -> |G | 6| -> |B | 1| ->

adj[B] -> |E | 4| -> |D | 2| -> |C | 1| -> |A | 1| ->

adj[C] -> |E | 4| -> |B | 1| ->

adj[D] -> |F | 1| -> |E | 2| -> |B | 2| ->

adj[E] -> |L | 4| -> |F | 2| -> |G | 1| -> |D | 2| -> |C | 4| -> |B | 4| ->

adj[F] -> |L | 2| -> |E | 2| -> |D | 1| -> |A | 2| ->

adj[G] -> |L | 5| -> |J | 1| -> |H | 3| -> |E | 1| -> |A | 6| ->

adj[H] -> |I | 2| -> |G | 3| ->

adj[I] -> |K | 1| -> |H | 2| ->

adj[J] -> |K | 1| -> |L | 3| -> |M | 2| -> |G | 1| ->

adj[K] -> |J | 1| -> |I | 1| ->

adj[L] -> |M | 1| -> |J | 3| -> |G | 5| -> |F | 2| -> |E | 4| ->

adj[M] -> |L | 1| -> |J | 2| ->

Depth First Graph Traversal

Starting with Vertex L

DF visited vertex L on edge @--L

DF visited vertex M on edge L--M

DF visited vertex J on edge M--J

DF visited vertex K on edge J--K

DF visited vertex I on edge K--I

DF visited vertex H on edge I--H

DF visited vertex G on edge H--G

DF visited vertex E on edge G--E

DF visited vertex F on edge E--F

DF visited vertex D on edge F--D

DF visited vertex B on edge D--B

DF visited vertex C on edge B--C

DF visited vertex A on edge B--A

Vertex	Discovery	Finish	Parent
--------	-----------	--------	--------

A	14	15	B
---	----	----	---

B	11	16	D
---	----	----	---

C	12	13	B
---	----	----	---

D	10	17	F
---	----	----	---

E	8	19	G
---	---	----	---

F	9	18	E
---	---	----	---

G	7	20	H
---	---	----	---

H	6	21	I
---	---	----	---

I	5	22	K
---	---	----	---

J	3	24	M
---	---	----	---

K	4	23	J
---	---	----	---

L	1	26	@
---	---	----	---

M	2	25	L
---	---	----	---

Breadth First Graph Traversal

Starting with Vertex L

BF visited vertex L on edge @--L

BF visited vertex M on edge L--M

BF visited vertex J on edge L--J

BF visited vertex G on edge L--G

BF visited vertex F on edge L--F

BF visited vertex E on edge L--E

BF visited vertex K on edge J--K

BF visited vertex H on edge G--H

BF visited vertex A on edge G--A

BF visited vertex D on edge F--D

BF visited vertex C on edge E--C

BF visited vertex B on edge E--B

BF visited vertex I on edge K--I

Vertex	Distance-from-L	Parent
--------	-----------------	--------

A	2	G
---	---	---

B	2	E
---	---	---

C	2	E
---	---	---

D	2	F
---	---	---

E	1	L
---	---	---

F	1	L
---	---	---

G	1	L
---	---	---

H	2	G
---	---	---

I	3	K
---	---	---

J	1	L
---	---	---

K	2	J
---	---	---

L	0	@
---	---	---

M	1	L
---	---	---

MST_Prim starting from vertex L

Vertex	Key	Parent
--------	-----	--------

L	0	@
---	---	---

M	1	L
---	---	---

F	2	L
---	---	---

D	1	F
---	---	---

J	2	M
---	---	---

K	1	J
---	---	---

G	1	J
---	---	---

I	1	K
---	---	---

E	1	G
---	---	---

B	2	D
---	---	---

C	1	B
---	---	---

A	1	B
---	---	---

H	2	I
---	---	---

MST edges:

B - A

D - B

B - C

F - D

G - E

L - F

J - G

I - H

K - I

M - J

J - K

L - M

Total MST Weight = 16

=====

Time taken: 172 ms

Memory used: 0 MB

=====

Kruskal.java:

Enter graph filename: Wgraph1.txt

Parts[] = 13 22 [plain format]

Reading edges from text file

Edge A--(1)--B

Edge A--(6)--G

Edge A--(2)--F

Edge B--(1)--C

Edge B--(2)--D

Edge B--(4)--E

Edge C--(4)--E

Edge D--(2)--E

Edge D--(1)--F

Edge E--(1)--G

Edge E--(2)--F

Edge E--(4)--L

Edge F--(2)--L

Edge G--(3)--H

Edge G--(1)--J

Edge G--(5)--L

Edge H--(2)--I

Edge I--(1)--K

Edge J--(2)--M

Edge J--(3)--L

Edge J--(1)--K

Edge L--(1)--M

MST_Kruskal

Sorted edges:

A --(1)-- B

B --(1)-- C

D --(1)-- F

E --(1)-- G

G --(1)-- J

I --(1)-- K

J --(1)-- K

L --(1)-- M

A --(2)-- F

E --(2)-- F

F --(2)-- L

B --(2)-- D

H --(2)-- I

J --(2)-- M

D --(2)-- E

G --(3)-- H

J --(3)-- L

B --(4)-- E

C --(4)-- E

E --(4)-- L

G --(5)-- L

A --(6)-- G

Step-by-step:

Step 1: A --(1)-- B find(A)=A find(B)=B

--> ADDED (MST weight so far = 1)

Step 2: B --(1)-- C find(B)=A find(C)=C

--> ADDED (MST weight so far = 2)

Step 3: D --(1)-- F find(D)=D find(F)=F

--> ADDED (MST weight so far = 3)

Step 4: E --(1)-- G find(E)=E find(G)=G

--> ADDED (MST weight so far = 4)

Step 5: G --(1)-- J find(G)=E find(J)=J

--> ADDED (MST weight so far = 5)

Step 6: I --(1)-- K find(I)=I find(K)=K

--> ADDED (MST weight so far = 6)

Step 7: J --(1)-- K find(J)=E find(K)=I

--> ADDED (MST weight so far = 7)

Step 8: L --(1)-- M find(L)=L find(M)=M

--> ADDED (MST weight so far = 8)

Step 9: A --(2)-- F find(A)=A find(F)=D

--> ADDED (MST weight so far = 10)

Step 10: E --(2)-- F find(E)=E find(F)=A

--> ADDED (MST weight so far = 12)

Step 11: F --(2)-- L find(F)=E find(L)=L

--> ADDED (MST weight so far = 14)

Step 12: B --(2)-- D find(B)=E find(D)=E

--> REJECTED (same component, would form a cycle)

Step 13: H --(2)-- I find(H)=H find(I)=E

--> ADDED (MST weight so far = 16)

Total MST Weight = 16

7. Real World Graph

For this I used the New York city road network from the 9th DIMACS Implementation Challenge to test the algorithms in a real-world setting. I chose this because it is a widely respected benchmark dataset from the Sapienza University of Rome and it is freely available unlike other datasets I tried to get. The link for the dataset is shown below

<http://www.diag.uniroma1.it/challenge9/download.shtml>

USA-road-d.NY.gr.gz

The dataset has 264346 edges (road intersections) and 733846 edges (road segments) and its in the DIMACS .gr format which I made sure algorithms accepted it. The Prims

algorithm took around 4 secs to complete and around 400 MB over multiple runs of the algorithm, but the Kruskal algorithm took around 4 secs and took 320 MB of memory.

Both programs read the DIMACS .gr format just fine, the constructor I made detects the starting lines with 'c' comments and read V and E from the 'p sp V E' line and reads edges from lines starting with 'a'. both algorithms agreed on the total MST weight even with such a big dataset.

8. Reflection

The assignment taught me how much the choice of data structures affects performance. Using an adjacency list instead of using the traditional matrix made the transversal fast and used less memory. Using the min-heap for prims showed me why decrease Key needs a reverse index as without it the algorithm would be too slow a large real-world graph like the example I used. Overall, I learned MST concepts and properties the differences between the two algorithms and edge lists vs adjacency list, the biggest problem was to understand the heaps and union find and to use the DIMACS .gr file type.